

Data Science শেখার যাত্রা

লেসন ১ থেকে লেসন ১২

Python • NumPy • Pandas • Statistics • Probability

প্রস্তুতকারী: Redwan

তারিখ: অক্টোবর ২০২৬

সূচিপত্র

পার্ট ১: প্রোগ্রামিং ফাউন্ডেশন

লেসন ১	Python বেসিক (variables, data types)
লেসন ২	if-else, loops, functions
লেসন ৩	NumPy — array ও numerical computing
লেসন ৪	Pandas — DataFrame বেসিক
লেসন ৫	CSV ফাইল লোড ও এক্সপোর্ট
লেসন ৬	ডেটা ক্লিনিং (missing values, duplicates)
লেসন ৭	groupby, sort_values, merge
লেসন ৮	মিনি প্রজেক্ট: COVID ডেটা বিশ্লেষণ

পার্ট ২: গণিত ও পরিসংখ্যান

লেসন ৯	Descriptive Statistics (mean, median, mode, variance)
লেসন ১০	Probability বেসিক
লেসন ১১	Distributions (Normal, Binomial)
লেসন ১২	Hypothesis Testing ও p-value

১ Python বেসিক — শুরুর ধাপ

কেন Python দিয়ে শুরু?

ডেটা সায়েন্সে Python ব্যবহার হয় কারণ এতে NumPy, Pandas, Scikit-learn, TensorFlow-এর মতো শক্তিশালী লাইব্রেরি আছে যেগুলো দিয়ে খুব সহজে ডেটা নিয়ে কাজ করা যায়।

১. Variables (ভ্যারিয়েবল)

ভ্যারিয়েবল হলো একটা "বাক্স" যেখানে তুমি ডেটা রাখো।

```
name = "Redwan"      # স্ট্রিং (টেক্সট)
age = 31              # ইন্টিজার (পূর্ণ সংখ্যা)
height = 5.9          # ফ্লোট (দশমিক সংখ্যা)
is_learning = True    # বুলিয়ান (True/False)
```

২. Basic Data Types

- **int** → পূর্ণ সংখ্যা (1, 2, 100)
- **float** → দশমিক সংখ্যা (3.14, 2.5)
- **str** → টেক্সট ("Hello")
- **bool** → True অথবা False
- **list** → একাধিক জিনিসের তালিকা [1, 2, 3]
- **dict** → key-value জোড়া {"name": "Redwan", "age": 31}

৩. একটা ছোট প্র্যাকটিস কোড

```
scores = [85, 90, 78, 92, 88]

total = sum(scores)
average = total / len(scores)

print("মোট নম্বর:", total)
print("গড় নম্বর:", average)
```

এই কোডটা রান করলে দেখবে — এটাই ডেটা সায়েন্সের প্রথম ধাপ: সংখ্যা নিয়ে বিশ্লেষণ করা!

হোমওয়ার্ক

1. Google Colab (colab.research.google.com) ব্যবহার করো, ইনস্টল লাগে না।
2. উপরের কোডটা রান করে দেখো।

3. `scores` লিস্টে নিজের পছন্দমতো ৫টা সংখ্যা বসিয়ে আবার গড় বের করো।

২ Logic ও Loops

১. if-else (শর্ত অনুযায়ী কাজ)

```
score = 85

if score >= 90:
    print("A+ গ্রেড")
elif score >= 80:
    print("A গ্রেড")
else:
    print("আরো ভালো করতে হবে")
```

পাইথন উপর থেকে নিচে একটা একটা করে শর্ত চেক করে, প্রথম যেটা সত্যি হয় শুধু সেটাই রান করে, বাকিগুলো স্কিপ করে।

২. for loop (বারবার কাজ করানো)

```
scores = [85, 90, 78, 92, 88]

for score in scores:
    if score >= 90:
        print(score, "→ A+")
    else:
        print(score, "→ পাস")
```

Round	score এর মান	আউটপুট
১ম	85	85 → পাস
২য়	90	90 → A+
৩য়	78	78 → পাস
৪র্থ	92	92 → A+
৫ম	88	88 → পাস

৩. Functions (কাজ পুনরায় ব্যবহারের জন্য)

```
def grade_checker(score):
    if score >= 90:
        return "A+"
    elif score >= 80:
        return "A"
```

```
else:
    return "Needs Improvement"

print(grade_checker(85)) # আউটপুট: A
print(grade_checker(95)) # আউটপুট: A+
```

return vs print: `return` মানে function থেকে একটা মান বের করে আনা (যা পরে ব্যবহার করা যায়), আর `print` শুধু স্ক্রিনে দেখানো।

8. মিনি-প্রজেক্ট

```
students = {
    "Rahim": 78,
    "Karim": 92,
    "Salma": 85,
    "Nadia": 65
}

for name, score in students.items():
    grade = grade_checker(score)
    print(name, "-", score, "-", grade)
```

```
Rahim - 78 - Needs Improvement
Karim - 92 - A+
Salma - 85 - A
Nadia - 65 - Needs Improvement
```

হোমওয়ার্ক

1. `grade_checker` function-টা রান করে দেখো।
2. `students` ডিকশনারিতে নিজের নাম আর স্কোর যোগ করো।
3. **চ্যালেঞ্জ:** একটা function বানাও `average_score(scores_list)` যেটা একটা লিস্টের গড় রিটার্ন করবে।

৩ NumPy — সংখ্যা নিয়ে দ্রুত কাজ করা

কেন NumPy?

সাধারণ Python লিস্ট দিয়ে হাজার হাজার সংখ্যা নিয়ে কাজ করলে সময় বেশি লাগে। NumPy দিয়ে একই কাজ অনেক দ্রুত হয়।

১. Import ও Array বানানো

```
import numpy as np

scores = np.array([85, 90, 78, 92, 88])
print(scores)
```

২. লিস্ট vs Array

```
# সাধারণ লিস্টে loop লাগে:
normal_list = [85, 90, 78, 92, 88]
new_list = [x + 5 for x in normal_list]

# NumPy তে সরাসরি:
scores = np.array([85, 90, 78, 92, 88])
new_scores = scores + 5      # প্রতিটা সংখ্যায় একসাথে ৫ যোগ (vectorization)
```

৩. পরিসংখ্যান ফাংশন

```
scores = np.array([85, 90, 78, 92, 88])

print("গড়:", np.mean(scores))
print("সর্বোচ্চ:", np.max(scores))
print("সর্বনিম্ন:", np.min(scores))
print("স্ট্যান্ডার্ড ডিভিয়েশন:", np.std(scores))
print("সর্টেড:", np.sort(scores))
```

৪. 2D Array (ম্যাট্রিক্স) ও axis

```
marks = np.array([
    [85, 90, 78],
    [92, 88, 95],
    [70, 65, 80]
])

print(np.mean(marks, axis=1)) # প্রতি রো (স্টুডেন্ট)-এর গড়
```

```
print(np.mean(marks, axis=0)) # প্রতি কলাম (সাবজেক্ট)-এর গড়  
print(np.mean(marks))       # axis ছাড়া — সব সংখ্যার overall গড়
```

axis=0 মানে "উপর থেকে নিচে" (কলাম বরাবর), axis=1 মানে "বাম থেকে ডানে" (রো বরাবর)। axis না দিলে পুরো array-কে ফ্ল্যাটেন করে একটাই গড় বের হয়।

হোমওয়ার্ক

1. `import numpy as np` লিখে NumPy চালু করো।
2. নিজের একটা array বানিয়ে mean, max, min বের করো।
3. **চ্যালেঞ্জ:** একটা 2x3 ম্যাট্রিক্স বানাও এবং axis=0, axis=1 দুই রকম গড় বের করো।

৪ Pandas — রিয়েল ডেটাসেট নিয়ে কাজ করা

কেন Pandas?

NumPy দিয়ে শুধু সংখ্যা নিয়ে কাজ করা যায়, কিন্তু Pandas এক্সেল শিটের মতো টেবিল আকারে (DataFrame) ডেটা সাজিয়ে দেখায়।

১. DataFrame বানানো

```
import pandas as pd

data = {
    "Name": ["Rahim", "Karim", "Salma", "Nadia"],
    "Age": [22, 25, 21, 23],
    "Score": [78, 92, 85, 65],
    "City": ["Dhaka", "Chittagong", "Dhaka", "Sylhet"]
}

df = pd.DataFrame(data)
print(df)
```

২. ডেটা দেখা ও বোঝা

```
print(df.head())          # প্রথম ৫টা রো
print(df.shape)           # (rows, columns)
print(df.columns)         # সব কলামের নাম
print(df.describe())      # numeric কলামের mean, min, max, std
```

৩. কলাম/রো বের করা ও ফিল্টার

```
print(df["Name"])
print(df[["Name", "Score"]])
print(df.iloc[0])

high_scorers = df[df["Score"] > 80]
print(high_scorers)
```

৪. নতুন কলাম যোগ করা

```
df["Grade"] = df["Score"].apply(lambda x: "A" if x >= 80 else "B")
print(df)
```

হোমওয়ার্ক

1. ৫-৬ জনের নাম, বয়স, মার্কস দিয়ে একটা DataFrame বানাও।
2. `df.describe()` চালিয়ে দেখো।
3. **চ্যালেঞ্জ:** বয়স ২০-এর বেশি এমন ডেটা ফিল্টার করো।

৫ CSV ফাইল লোড ও এক্সপোর্ট

১. Colab-এ ফাইল আপলোড

```
from google.colab import files
uploaded = files.upload()
```

২. CSV ফাইল লোড করা

```
import pandas as pd

df = pd.read_csv("filename.csv")
print(df.head())
```

৩. ইন্টারনেট থেকে সরাসরি লোড

```
url = "https://raw.githubusercontent.com/datasets/covid-19/main/data/countries-aggregated.csv"
df = pd.read_csv(url)
print(df.head())
```

৪. ডেটাসেট সম্পর্কে দ্রুত জানা

```
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
```

৫. CSV আকারে সেভ করা

```
df.to_csv("my_output.csv", index=False)
```

হোমওয়ার্ক

- কোভিড ডেটাসেট URL দিয়ে `pd.read_csv()` করো।
- `df.shape`, `df.info()`, `df.isnull().sum()` চালাও।
- চ্যালেঞ্জ:** `df.head(10)` দিয়ে `head()` -এর পার্থক্য দেখো।

৬ ডেটা ক্লিনিং — Missing Values ও Duplicates

১. নোংরা ডেটাসেট

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Rahim", "Karim", "Salma", "Nadia", "Karim"],
    "Age": [22, 25, np.nan, 23, 25],
    "Score": [78, 92, 85, np.nan, 92]
}

df = pd.DataFrame(data)
```

২. Missing Value খুঁজে বের করা

```
print(df.isnull().sum())
```

৩. Missing Value হ্যান্ডেল করার ৩টা উপায়

উপায় ১ — মুছে ফেলা:

```
df_dropped = df.dropna()
```

উপায় ২ — গড় দিয়ে পূরণ (সবচেয়ে বেশি ব্যবহৃত):

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
df["Score"] = df["Score"].fillna(df["Score"].mean())
```

উপায় ৩ — নির্দিষ্ট মান দিয়ে পূরণ:

```
df["Age"] = df["Age"].fillna(0)
```

Age/Score-এর মতো কলামে ০ বসালে গড় নষ্ট হয়ে যায় (যেমন ০ বসালে গড় 19.0 হয়, কিন্তু mean বসালে বাস্তবসম্মত 23.75 থাকে)। তাই numeric কলামে সাধারণত mean ব্যবহার করা হয়, 0 শুধু তখনই ঠিক যখন 0 সত্যিই "কিছু নেই" বোঝায়।

8. Duplicate খুঁজে বের করা ও মোছা

```
print(df.duplicated())  
df_no_duplicates = df.drop_duplicates()
```

৫. Data Type ঠিক করা

```
print(df.dtypes)  
df["Age"] = df["Age"].astype(int)
```

হোমওয়ার্ক

1. নোংরা data dictionary কপি করে রান করো।
2. `dropna()` আর `fillna(mean)` — দুটো আলাদা করে try করো।
3. `drop_duplicates()` চালিয়ে duplicate সরাও।
4. **চ্যালেঞ্জ:** COVID ডেটাসেটে `df.duplicated().sum()` চালিয়ে দেখো।

৭ groupby, sort_values ও merge

১. groupby: গ্রুপ করে হিসাব করা

```
import pandas as pd

sales = pd.DataFrame({
    "City": ["Dhaka", "Dhaka", "Sylhet", "Sylhet", "Dhaka"],
    "Product": ["Pen", "Book", "Pen", "Book", "Pen"],
    "Amount": [100, 250, 80, 300, 120]
})

print(sales.groupby("City")["Amount"].sum())
```

City	
Dhaka	470
Sylhet	380

২. একাধিক হিসাব একসাথে

```
print(sales.groupby("City")["Amount"].agg(["sum", "mean", "max"]))
```

৩. sort_values: সাজানো

```
result = sales.groupby("City")["Amount"].sum()
print(result.sort_values(ascending=False))
```

৪. COVID ডেটাসেটে প্রয়োগ

COVID ডেটা cumulative (জমা হওয়া), তাই যোগ না করে প্রতি দেশের সর্বোচ্চ (শেষ দিনের) মান নিতে হবে:

```
url = "https://raw.githubusercontent.com/datasets/covid-19/main/data/countries-agggregated.csv"
df = pd.read_csv(url)

top10 = df.groupby("Country")
["Confirmed"].max().sort_values(ascending=False).head(10)
print(top10)
```

৫. merge: দুটো টেবিল জোড়া লাগানো

```
students = pd.DataFrame({"ID": [1, 2, 3], "Name": ["Rahim", "Karim", "Salma"]})
scores = pd.DataFrame({"ID": [1, 2, 3], "Score": [78, 92, 85]})

merged = pd.merge(students, scores, on="ID")
print(merged)
```

on="ID" মানে ID কলাম মিলিয়ে দুটো টেবিল এক করা — এটা এক্সেলের VLOOKUP-এর মতো কাজ করে। মনে রাখা জরুরি: উভয় DataFrame-এর সব কলামের length সমান হতে হবে, নইলে ValueError: All arrays must be of the same length আসবে।

হোমওয়ার্ক

1. `groupby("Product")["Amount"].sum()` চালাও।
2. COVID ডেটাসেটে top 10 দেশ বের করো (Confirmed দিয়ে)।
3. **চ্যালেঞ্জ:** সবচেয়ে বেশি Deaths হওয়া ১০টা দেশ বের করো।
4. **বোনাস:** `df[df["Country"] == "Bangladesh"].tail()` দিয়ে ফিল্টার করো।

৮ মিনি প্রজেক্ট — COVID Data Analysis

এ পর্যন্ত শেখা (Pandas, cleaning, groupby, filtering) সব একসাথে ব্যবহার করে একটা সম্পূর্ণ বিশ্লেষণ।

ধাপ ১: ডেটা লোড ও প্রাথমিক চেক

```
import pandas as pd

url = "https://raw.githubusercontent.com/datasets/covid-19/main/data/countries-aggregated.csv"
df = pd.read_csv(url)

print(df.shape)
print(df.isnull().sum())
print(df.duplicated().sum())
```

ধাপ ২: সবচেয়ে বেশি আক্রান্ত ১০টা দেশ

```
top10_confirmed = df.groupby("Country")
["Confirmed"].max().sort_values(ascending=False).head(10)
```

ধাপ ৩: সবচেয়ে বেশি মৃত্যু হওয়া ১০টা দেশ

```
top10_deaths = df.groupby("Country")
["Deaths"].max().sort_values(ascending=False).head(10)
```

ধাপ ৪: Recovery Rate বের করা

```
latest = df.groupby("Country").max()
latest["Recovery_Rate"] = (latest["Recovered"] / latest["Confirmed"]) * 100
print(latest[["Confirmed", "Recovered", "Recovery_Rate"]].sort_values("Recovery_Rate", ascending=False).head(10))
```

ধাপ ৫: বাংলাদেশ vs অন্য দেশ তুলনা

```
bd = df[df["Country"] == "Bangladesh"].tail(1)
india = df[df["Country"] == "India"].tail(1)
```

ধাপ ৬: ফলাফল CSV-তে সেভ করা

```
top10_confirmed.to_csv("top10_confirmed_countries.csv")
```

কাজ

1. প্রতিটা ধাপ একটা নতুন notebook-এ চালাও (নাম: lesson-08-covid-mini-project.ipynb)।
2. প্রতিটা ধাপের উপরে markdown cell-এ ব্যাখ্যা লেখো।
3. ফলাফল দেখে ২-৩ লাইনের সারাংশ লেখো।
4. পুরো notebook GitHub-এ আপলোড করো।

৯ Descriptive Statistics

১. Central Tendency

Mean (গড়):

```
scores = [70, 75, 80, 85, 200]
mean = sum(scores) / len(scores)
print(mean)    # 102.0
```

সমস্যা: একটা বড় সংখ্যা (200) পুরো গড়কে টেনে উপরে নিয়ে গেছে — একে বলে **outlier**-এর প্রভাব।

Median (মধ্যক):

```
import numpy as np
print(np.median(scores))    # 80.0
```

সংখ্যা সাজিয়ে মাকেরটা: 70, 75, **80**, 85, 200 — outlier-এ প্রভাবিত হয় না।

Mode:

```
from scipy import stats
data = [1, 2, 2, 3, 4, 2]
print(stats.mode(data))    # mode হলো 2
```

২. Spread/Dispersion

```
data = np.array([70, 75, 80, 85, 90])
print("Variance:", np.var(data))
print("Std Dev:", np.std(data))
```

```
class_A = np.array([78, 80, 82, 79, 81])    # কম ছড়ানো
class_B = np.array([50, 95, 60, 100, 45])    # বেশি ছড়ানো

print("Class A std:", np.std(class_A))
print("Class B std:", np.std(class_B))
```

৩. Percentile ও Quartile

```
data = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
print("25th percentile:", np.percentile(data, 25))
print("50th percentile:", np.percentile(data, 50))
print("75th percentile:", np.percentile(data, 75))
```

এই সংখ্যাগুলোই `df.describe()` -এ 25%, 50%, 75% রো-তে দেখা যায়।

হোমওয়ার্ক

1. COVID ডেটাসেট থেকে `df["Deaths"].mean()` ও `median()` তুলনা করো।
2. `df["Confirmed"].std()` বের করো।
3. **চ্যালেঞ্জ:** নিজের ৫-৭টা সংখ্যার 25/50/75 পার্সেন্টাইল বের করো।

১০ Probability (সম্ভাবনা) বেসিক

১. বেসিক সংজ্ঞা

$\text{Probability} = (\text{যা চাই সেটা ঘটায় উপায়}) / (\text{মোট সম্ভাব্য উপায়})$

```
# কয়েন টসে হেড আসার সম্ভাবনা
favorable = 1
total = 2
print(favorable / total)    # 0.5
```

Probability সবসময় 0 থেকে 1-এর মধ্যে থাকে।

২. Simulation দিয়ে যাচাই

```
import random

heads = 0
trials = 10000
for i in range(trials):
    if random.choice(["Head", "Tail"]) == "Head":
        heads += 1

print("হেড আসার শতাংশ:", (heads / trials) * 100)
```

৩. ডেটাসেট থেকে Probability

```
import pandas as pd
students = pd.DataFrame({
    "Name": ["Rahim", "Karim", "Salma", "Nadia", "Jamal"],
    "Pass": ["Yes", "No", "Yes", "Yes", "No"]
})

pass_count = (students["Pass"] == "Yes").sum()
probability_pass = pass_count / len(students)
print(probability_pass)    # 0.6
```

৪. Conditional Probability

```
data = pd.DataFrame({
    "Fever": ["Yes", "Yes", "No", "Yes", "No"],
    "Covid": ["Yes", "No", "No", "Yes", "No"]
})
```

```
fever_patients = data[data["Fever"] == "Yes"]
covid_given_fever = (fever_patients["Covid"] == "Yes").sum() / len(fever_patients)
print(covid_given_fever)
```

হোমওয়ার্ক

1. কয়েন টস simulation ১,০০,০০০ ট্রায়াল দিয়ে চালাও।
2. নিজের ৭-৮ জনের Pass/Fail ডেটা বানিয়ে probability বের করো।
3. **চ্যালেঞ্জ:** ডাইস ১০,০০০ বার simulate করে "৬" আসার শতাংশ (তাত্ত্বিক ১৬.৬৭%) যাচাই করো।

১১ Distributions — Normal ও Binomial

১. Normal Distribution (ঘণ্টা-আকৃতির বক্ররেখা)

```
import numpy as np
import matplotlib.pyplot as plt

heights = np.random.normal(loc=5.5, scale=0.3, size=10000)

plt.hist(heights, bins=50)
plt.title("উচ্চতার Distribution")
plt.show()
```

`loc` = mean, `scale` = standard deviation, `size` = কতজনের ডেটা।

২. 68-95-99.7 নিয়ম

পরিসীমা	ডেটার শতাংশ
± 1 std dev	৬৮%
± 2 std dev	৯৫%
± 3 std dev	৯৯.৭%

```
mean = np.mean(heights)
std = np.std(heights)
print(mean - std, "থেকে", mean + std) # ৬৮% ডেটা এখানে
```

৩. Binomial Distribution

```
from scipy import stats

n = 10 # মোট ট্রায়াল
p = 0.5 # প্রতিবার হেড আসার সম্ভাবনা

prob_5_heads = stats.binom.pmf(5, n, p)
print(prob_5_heads)
```

```
# MCQ গেসিং উদাহরণ (১০টা প্রশ্ন, ৪টা অপশন)
n, p = 10, 0.25
for correct in range(0, 11):
    print(correct, stats.binom.pmf(correct, n, p))
```

৪. কেন ML-এ দরকার?

- Linear Regression ধরে নেয় error Normal distribution মেনে চলে।
- Binomial distribution ব্যবহার হয় yes/no ধরনের Classification সমস্যায়।

হোমওয়ার্ক

1. heights হিস্টোগ্রাম দেখো — bell-shape যাচাই করো।
2. 68-95-99.7 নিয়ম নিজে হাতে যাচাই করো।
3. চ্যালেঞ্জ: কয়েন ২০ বার টসে ঠিক ১০ বার হেড আসার সম্ভাবনা বের করো।

১২ Hypothesis Testing ও p-value

১. দুটো Hypothesis

- **Null Hypothesis (H0):** "কোনো পার্থক্য নেই"
- **Alternative Hypothesis (H1):** "পার্থক্য আছে"

২. p-value কী?

p-value বলে: "যদি H0 সত্যি হতো, তাহলে এই রেজাল্ট (বা আরও extreme কিছু) পাওয়ার সম্ভাবনা কত?"

p-value	সিদ্ধান্ত
$p < 0.05$	পার্থক্য/প্রভাব আছে (H0 বাতিল)
$p \geq 0.05$	যথেষ্ট প্রমাণ নেই (H0 বাতিল করা যাচ্ছে না)

৩. t-test উদাহরণ

```
from scipy import stats
import numpy as np

old_method = np.array([65, 70, 68, 72, 69, 71, 66])
new_method = np.array([75, 78, 80, 76, 82, 79, 77])

t_stat, p_value = stats.ttest_ind(old_method, new_method)
print("p-value:", p_value)

if p_value < 0.05:
    print("পার্থক্যটা statistically significant")
else:
    print("যথেষ্ট প্রমাণ নেই")
```

৪. Coin Fairness Test

```
from scipy.stats import binomtest

result = binomtest(65, n=100, p=0.5)
print("p-value:", result.pvalue)
```

সতর্কতা: $p < 0.05$ মানে এই না যে পার্থক্যটা অবশ্যই বড়/গুরুত্বপূর্ণ — এটা শুধু বলে পার্থক্যটা কাকতালীয় না হওয়ার সম্ভাবনা বেশি। "statistically significant" আর "practically important" সবসময় এক জিনিস না।

হোমওয়ার্ক

1. `old_method` vs `new_method` t-test চালাও।
2. নিজের দুটো গ্রুপের ডেটা বানিয়ে t-test চালাও।
3. **চ্যালেঞ্জ:** coin test-এ $n=1000$, $\text{heads}=650$ দিয়ে p-value-এর পরিবর্তন লক্ষ্য করো।